👩‍🎓 **STUDENT TABLE**

Rno (PK)	Sname	cid (Fk)	marks
12	Harsha	1	99
14	Rohith	4	95
15	Shiva	4	91
16	Rahul	4	90
83	Arun	1	97
32	SNR	3	73
18	Sri	2	70
1	Krishna	NULL	NULL
2	Arjuna	NULL	NULL
10	Dheeraj	5	34

📚 COURSES TABLE

Course ID (PK)	cname	tot
1	CSE	100
2	ECE	75
3	EEE	75
4	AIML	100
5	CIVIL	100
6	MECH	100
7	DS	100

🔹 INNER JOIN

An **INNER JOIN** returns only the rows where there is a match in both tables.

So, only students who are enrolled in a valid course (cid matches courseid) will be shown.

👉 In this case, students with course IDs **1, 4, 3, 2, 5** are included.
👉 Students with **NULL (no course)** are excluded.

✅ **Students displayed:**

Harsha, Rohith, Shiva, Rahul, Arun, SNR, Sri, Dheeraj

❌ **Not displayed:**

Krishna, Arjuna (because they have no course)

💻 **Query**

```
SELECT *
FROM student AS S
INNER JOIN courses AS C
ON S.cid = C.courseid;
```

🔹 LEFT JOIN

A **LEFT JOIN** returns:

- All records from the **left table (student)**
- Matching records from the **right table (courses)**

- If no match → values from the **courses table become NULL**

📌 **What happens here:**

- Every student is shown
- If a student has **no course (cid = NULL)** → course details will be **NULL**

✅ **Students displayed:**

Harsha, Rohith, Shiva, Rahul, Arun, SNR, Sri, Dheeraj, Krishna, Arjuna

🔍 **Special cases:**

- **Krishna, Arjuna** → no course → course columns = NULL

💻 **Query**

```
SELECT *
FROM student AS S
LEFT JOIN courses AS C
ON S.cid = C.courseid;
```

🔹 **RIGHT JOIN**

A **RIGHT JOIN** returns:

- All records from the **right table (courses)**
- Matching records from the **left table (student)**
- If no match → values from the **student table become NULL**

📌 **What happens here:**

- Every course is shown
- If a course has **no students enrolled** → student details will be **NULL**

✅ **Courses always displayed:**

CSE, ECE, EEE, AIML, CIVIL, MECH, DS

🔍 **Special cases:**

- **MECH, DS** → no students → student columns = NULL

💻 **Query**

```
SELECT *
FROM student AS S
RIGHT JOIN courses AS C
ON S.cid = C.courseid;
```

🧠 FULL JOIN or FULL OUTER JOIN

👉 **There is NO difference**

- FULL JOIN = FULL OUTER JOIN
- Both mean the **same thing**
- OUTER keyword is **optional**

✅ Example (both are equal):

```
SELECT * FROM student S
FULL JOIN courses C
ON S.cid = C.courseid;
```

```
SELECT * FROM student S
FULL OUTER JOIN courses C
ON S.cid = C.courseid;
```

📘 **What FULL OUTER JOIN Does**

It returns **everything from both tables**:

✓ **Includes:**

- Matching rows (student + course)
- Students with no course → course = NULL
- Courses with no students → student = NULL

🔍 **Based on Your Data**

✅ **Matched:**

Harsha, Rohith, Shiva, Rahul, Arun, SNR, Sri, Dheeraj

❌ **Left-only (no course):**

Krishna, Arjuna

❌ **Right-only (no students):**

MECH, DS

📊 **Output Understanding**

Student	Course	Result
Match exists	Match exists	Normal row
Student only	No course	Course = NULL
Course only	No student	Student = NULL

💻 Syntax (Standard SQL)

```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON condition;
```

⚠️ Important (Very Common Interview Point)

👉 Not all databases support FULL JOIN directly

❌ Not supported in:

- MySQL

✅ Supported in:

- PostgreSQL
- SQL Server
- Oracle

🔧 Alternative (for MySQL)

```
SELECT *
FROM student S
LEFT JOIN courses C
ON S.cid = C.courseid

UNION

SELECT *
FROM student S
RIGHT JOIN courses C
ON S.cid = C.courseid;
```

⚡ Key Differences Summary

Feature	FULL JOIN	FULL OUTER JOIN
Meaning	Same	Same
OUTER keyword	Optional	Explicit
Result	All rows from both tables	Same
Usage	Short form	Full form

🧠 One-Line Memory Trick

👉 FULL JOIN = LEFT JOIN + RIGHT JOIN (without duplicates)

🧠 JOIN + NULL Filtering

🔹 LEFT JOIN + WHERE NULL

👉 Finds **students with no course**

```
SELECT *
FROM student S
LEFT JOIN courses C
ON S.cid = C.courseid
WHERE C.courseid IS NULL;
```

✔ Returns: Krishna, Arjuna
✔ Logic: no match → right table becomes NULL

🔹 RIGHT JOIN + WHERE NULL

👉 Finds **courses with no students**

```
SELECT *
FROM student S
RIGHT JOIN courses C
ON S.cid = C.courseid
WHERE S.cid IS NULL;
```

✔ Returns: MECH, DS
✔ Logic: no match → left table becomes NULL

🧠 SELF JOIN (Employee → Manager)

👨‍💼 EMPLOYEE TABLE (Hierarchy View)

id	name	manager_id
103	casey	NULL
104	donald	103
102	bob	104
101	adam	103

🔹 Query

```
SELECT Emp2.name AS EmployeeName,
       Emp1.name AS ManagerName
FROM Employee Emp1
JOIN Employee Emp2
ON Emp1.id = Emp2.manager_id;
```

✔ Emp1 → Manager
✔ Emp2 → Employee

🧠 UNION vs UNION ALL

🔹 UNION

```
SELECT * FROM Employee
UNION
SELECT * FROM Employee;
```

✔ Removes duplicates
✔ Output = only unique rows

🔹 UNION ALL

```
SELECT * FROM Employee
UNION ALL
SELECT * FROM Employee;
```

✔ Keeps duplicates
✔ Output = rows repeated twice

⚡ Key Differences

Feature	UNION	UNION ALL
Duplicates	Removed	Allowed
Performance	Slower	Faster
Use case	Clean unique data	Full data (including repeats)

🧠 Quick Memory

- LEFT + NULL → missing in right
- RIGHT + NULL → missing in left
- SELF JOIN → same table relation
- UNION → unique
- UNION ALL → all rows